

1 Отчёт по задаче 3.2: синтаксические шаблоны вместо нейросети

1.1 Проверка метрик нейросети

В `relation_matcher/README.md` написано — ≈ 0.58 accuracy / 0.41 macro-F1, полученное будто бы через `eval_holdout.py`. При проверке фактический файл `relation_matcher/holdout_metrics.json` содержит вот такие значения:

```
{
  "per_pair_accuracy": 0.302,
  "per_pair_macro_f1": 0.250
}
```

То есть README и committed JSON-файл с метриками расходятся; я взял за основу файл метрик, а не README.

1.2 Откуда взяты шаблоны

Шаблоны выведены из `reference_chunk` (`chunks_ontology_text_reference.jsonl`), сгруппированных по типу связи. При разборе обнаружилась проблема, которая определила результат всего эксперимента.

Input и Output неразличимы по тексту. Одна и та же пара может встречаться в эталоне с обоими типами связи, с идентичным `reference_chunk`, только с переставленными местами `concept_a/concept_b`. Пример — пара «деление с остатком» / «натуральное число» на тексте *«Арифметика... включающая операции сложения, умножения, вычитания и деления с остатком, причём носителем является множество натуральных чисел и ноль»*: в одной записи это `Output`, в другой (с переставленными понятиями) — `Input`. Получается, что никакой синтаксический шаблон не может различить эти два класса на таком тексте — шаблон видит только текст, а не то, какое понятие подставлено в `concept_a`, а какое в `concept_b`. Это задокументировано тестом `test_input_output_are_not_distinguishable_by_template_on_identical_text`.

Association и Instance почти не имеют собственных маркеров. В просмотренных примерах оба типа вводятся тем же оборотом «X называется/является Y», что и `generalization` (самый частый класс, 188/372) — специфичных фраз, которые встречались бы заметно чаще именно в этих классах, найти не удалось.

Composition, aggregation, dependency, generalization Итоговые шаблоны — `templates.yaml`.

1.3 Механизм сопоставления

`relation_templates.py`, класс `TemplateRelationClassifier`. Для пары (`concept_a`, `concept_b`, `reference_chunk`) строится отдельный маленький `AstDocument` из одного этого куска (не из всего учебника заново — иначе на 372 парах и ~ 35 шаблонах это было бы слишком медленно), и для каждого типа связи выполняется DSL-запрос на каждый его шаблон:

```
CONTEXT sentence[<=20]
FOR a: /regex_concept_a/i, b: /regex_concept_b/i, tpl: /regex_phrase/i
RETURN count
```

Берется тип связи, у которого сработало больше всего *разных* шаблонов (при равенстве — по LABEL_PRIORITY, специфичные типы впереди **generalization**); если не сработал ни один — откат на **generalization**. Голосование по числу сработавших шаблонов, а не первый совпавший по приоритету — так изначально не делали. Регэкспы понятий строятся той же функцией `concept_regex`, что и в `build_chunks_dataset.py`.

1.4 Голосование по числу шаблонов

Первая версия брала первый сработавший шаблон в порядке приоритета типов. Заменял на голосование: считаем для каждого типа число разных сработавших шаблонов, берём тип с максимумом. Заодно добавил ещё несколько шаблонов для **composition/aggregation/dependency**, выведенных из полного списка примеров каждого класса — «модулем над», «строится как» (**composition**), «делится на», «часть массива/отрезка» (**aggregation**), «определяется тем», отрицательный, чтобы «не зависит от» (обратное утверждение) не засчитывалось как **dependency**.

Эффект: accuracy 0.446 → 0.454, macro-F1 0.124 → 0.136.

1.5 Результаты эксперимента

Все варианты прогнаны на одних и тех же 372 парах из `chunks_ontology_text_reference.jsonl`

Подход	Accuracy	Macro-F1
Нейросеть, честная оценка (<code>holdout_metrics.json</code>)	0.302	0.250
Всегда generalization (majority-class baseline)	0.505	≈0.075
Синтаксические шаблоны, первая версия (приоритет)	0.446	0.124
Синтаксические шаблоны, голосование + расширенные фразы	0.454	0.136